Introduction to Containers
IEM 4723 Information Systems Design

Dr. Paritosh Ramanan

School of Industrial Engineering and Management
Oklahoma State University

Fall 2026

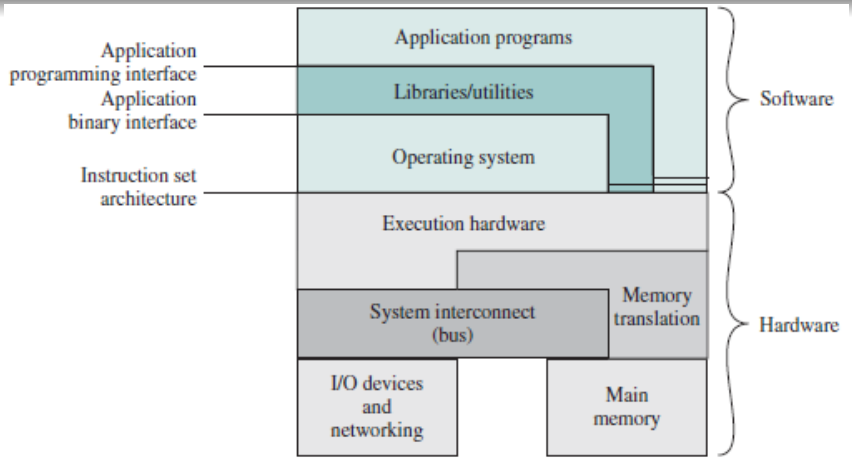
What is containerization?

How do computer programs work?

"OS driven Application Development" Operating Systems - Internals and Design Principles 7th ed - W. Stallings (Pearson, 2012)

- An OS has multiple layers of abstraction to allow smooth application development.
- A developer only needs to install "high level" libraries and utilities to develop apps.
- The OS takes care of everything under the hood to provide a seamless user experience.

What is happening under the hood?

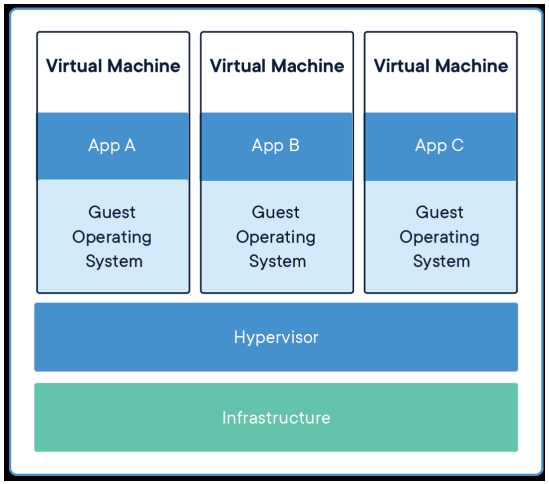


Simple python program

```
import numpy as np
a = np.array([1,2,3,4])
b = np.mean(a)
print(b)
```

- The python interpreter searches library dependencies in the installed libraries/utilities.
- Upon success, the relevant modules are imported in program runtime.
- The program logic statements are executed sequentially.
- The OS takes care of all the internals
  - How to interface with the processor for the math operation (addition for the mean).
  - Where to store the result?
  - How to interface with your monitor to "print" your command.

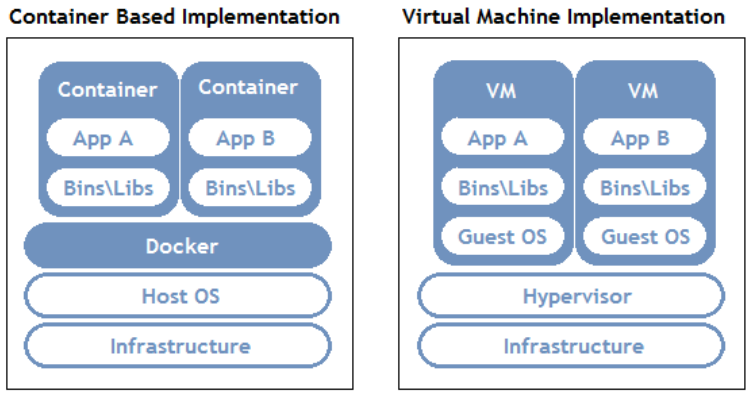
What are Virtual Machines (VM)?



https://www.docker.com/

- Since the early 2000s, there has been a push to squeeze more compute on physical machines
- Led to virtualization, wherein multiple host OS can be run on one physical system.
- On your Windows computer, download VMWare and now you can run Linux, AIX, HP-UX, BSD, FreeBSD etc.
- You can even run MacOS as well.

What are containers?

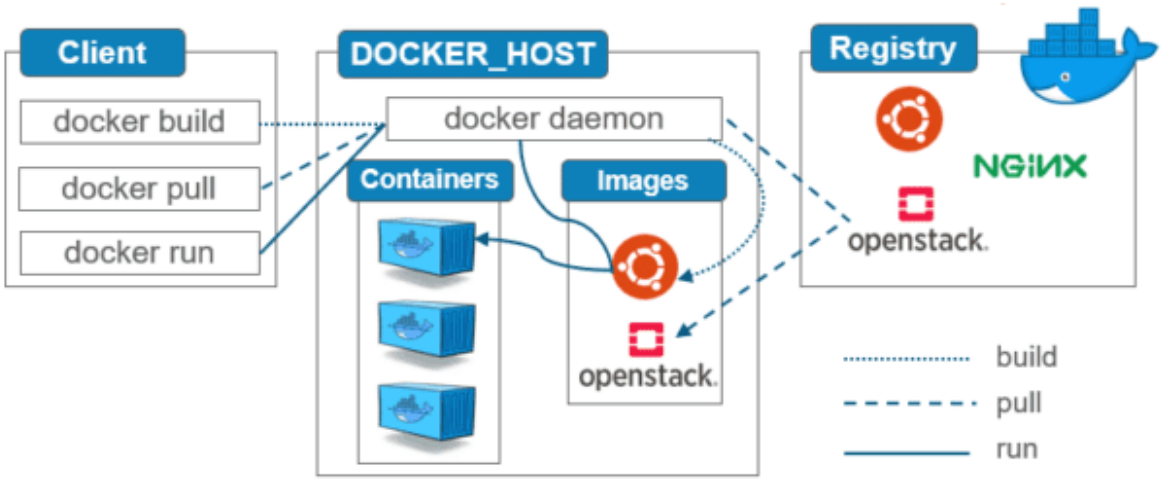


https://www.aquasec.com/cloud-native-academy/docker-container/docker-architecture/

- Containers go one step further by presenting a deeper level of virtualization.
- Abstracts away the painful process of installing and keeping track of system dependencies on host system.
- Allows rapid prototyping and roll out of software features faster.
- Docker and Podman are the popular choices.

How do containers work?

How do containers work?



https://geekflare.com/docker-architecture/

- Install docker daemon on development machines.
- Use Docker API to create container images and deploy them to a registry.
- These images can be used by anyone on any machine that supports docker.

Benefits of Containerization

Benefits of Containerization

Containerization enables the following capabilities.

- Sharing and running code without the pain of installing any dependencies.
- All software dependencies to be packaged into a container image by the research owners.
- Seamless execution of their container images by others.
- Layering of software frameworks without disruption.
- Support across any operating system! Linux, Windows even Unix!
- Container images are quick and easy to build and are the weapon of choice in DevOps Engineering!
- Popular containerization tools include Docker and Podman, which are mutually compatible 100%

Containerization Basics

Some Container Principles

- Container Creation
  - Hub and Base Images
- Container Repository
  - Create, Store and Share your code.
- Container Handling
  - Run and stop containers.
- Container Development
  - Debug, layer and reuse your code.

Docker Configuration and Install

- Download the docker client
  - `https://www.docker.com/products/docker-desktop`
- Install docker client
  - Make sure you have command line access.
- Create an account
  - The username and password is going to be required later.
- Windows users should rely on WSL (Windows Subsystem for Linux) or the Windows command line
  - `https://learn.microsoft.com/en-us/windows/wsl/install`
- Windows users, if you're unable to install Docker, please let me know as soon as possible.

How to create Docker containers?

Directory structure
- /path/to/code/
  - Dockerfile
  - requirements.txt
  - app.py

Dockerfile

```
FROM python:3.7-alpine
WORKDIR /code
RUN apk add --no-cache gcc musl-dev linux-headers
COPY /path/to/code/ .
RUN pip install -r requirements.txt
CMD ["python app.py"]
```

Docker image name to use as base

Change directory to location of code on the image

Install dependencies if needed (not contained in base image)

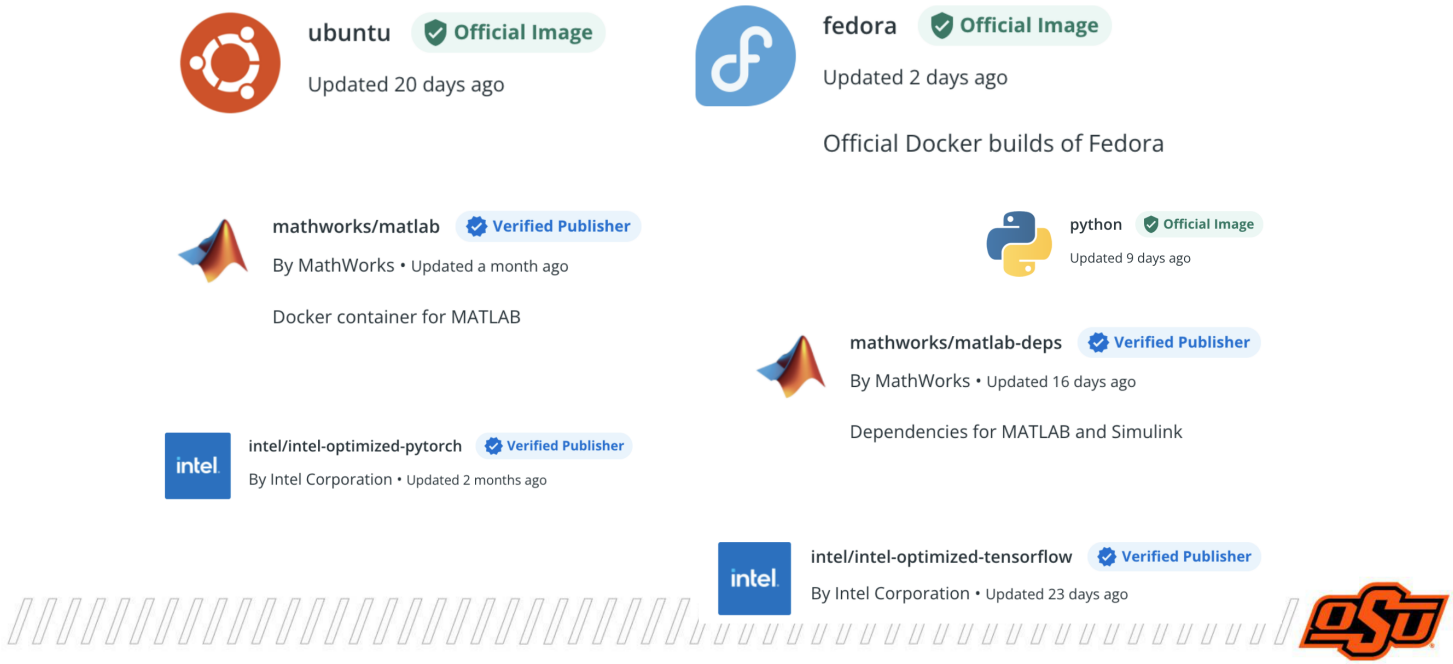
Install python dependencies for your program

Copy code to correct location on the image

Command to run your code

- Create the image by running `docker build -t <app_name> .`
- Run the image by executing `docker run -d <app_name>`

Some base image examples



Explore more images at `https://hub.docker.com`

Creating a Docker Image

Execute these commands from `/path/to/code/`

- Run a simple build using

```
docker build .
```

Creating a Docker Image

Execute these commands from `/path/to/code/`

- Run a simple build using

  ```
  docker build .
  ```

- Build and tag the image using

  ```
  docker build -t home_example:1.0 .
  ```

Creating a Docker Image

Execute these commands from `/path/to/code/`

- Run a simple build using

  ```
  docker build .
  ```

- Build and tag the image using

  ```
  docker build -t home_example:1.0 .
  ```

- Inspect the image using

  ```
  docker image inspect home_example:1.0
  ```

Some Container Principles

- Container Creation
  - Hub and Base Images
- Container Repository
  - Create, Store and Share your code.
- Container Handling
  - Run and stop containers.
- Container Development
  - Debug, layer and reuse your code.

Pushing to a Repository

Execute these commands from `/path/to/code/`

- Tag the image using

```
docker tag home_example:1.0 pramanan3/home_example:
    latest
```

Pushing to a Repository

Execute these commands from `/path/to/code/`

- Tag the image using

```
docker tag home_example:1.0 pramanan3/home_example:
    latest
```

- Push the image to a repository using

```
docker push pramanan3/home_example:latest
```

Pushing to a Repository

Execute these commands from `/path/to/code/`

- Tag the image using

  ```
  docker tag home_example:1.0 pramanan3/home_example:latest
  ```

- Push the image to a repository using

  ```
  docker push pramanan3/home_example:latest
  ```

- Remember!
  - Registry is a service for many images and projects, such as `hub.docker.com`
  - Repository is a collection of versions of the same image, such as the `home_example` repository, which contains `home_example:latest`, `home_example:v1.0`, and `home_example:debug`
  - A registry stores a collection of repositories.
  - Lets look at the registry and its repository we just used!

Some Container Principles

- Container Creation
  - Hub and Base Images
- Container Repository
  - Create, Store and Share your code.
- Container Handling
  - Run and stop containers.
- Container Development
  - Debug, layer and reuse your code.

Different Ways to Run a Container

- Running a container
  - A simple one-time run uses `docker run home_example:1.0`
  - Running as a daemon uses `docker run -d home_example:1.0`

Different Ways to Run a Container

- Running a container
  - A simple one-time run uses `docker run home_example:1.0`
  - Running as a daemon uses `docker run -d home_example:1.0`
- Name Container as Daemon
  - `docker run -d --name create_daemon_ex home_example:1.0`

Different Ways to Run a Container

- Running a container
  - A simple one-time run uses `docker run home_example:1.0`
  - Running as a daemon uses `docker run -d home_example:1.0`
- Name Container as Daemon
  - `docker run -d --name create_daemon_ex home_example:1.0`
- Here is a common gotcha that will fail. Why?
  - `docker run -d --name create_daemon_ex2 home_example:2.0`

Different Ways to Run a Container

- Running a container
  - A simple one-time run uses `docker run home_example:1.0`
  - Running as a daemon uses `docker run -d home_example:1.0`
- Name Container as Daemon
  - `docker run -d --name create_daemon_ex home_example:1.0`
- Here is a common gotcha that will fail. Why?
  - `docker run -d --name create_daemon_ex2 home_example:2.0`
- Checking on containers
  - List running containers using `docker ps`
  - List all containers using `docker container ls -all`

Different Ways to Run a Container

- Running a container
  - A simple one-time run uses `docker run home_example:1.0`
  - Running as a daemon uses `docker run -d home_example:1.0`
- Name Container as Daemon
  - `docker run -d --name create_daemon_ex home_example:1.0`
- This will fail. Why?
  - `docker run -d --name create_daemon_ex home_example:1.0`
- Checking on containers
  - List running containers using `docker ps`
  - List all containers using `docker container ls -a`
- Stop, remove, and rerun the container
  - `docker stop create_daemon_ex`
  - `docker rm create_daemon_ex`
  - `docker run -d --name create_daemon_ex home_example:2.0`
- SUCCESS!

Some Container Principles

- Container Creation
  - Hub and Base Images
- Container Repository
  - Create, Store and Share your code.
- Container Handling
  - Run and stop containers.
- Container Development
  - Debug, layer and reuse your code.

Debugging Containers

Let's use a different Dockerfile.

- `Dockerfile.debug`
- Note the difference! `CMD tail -f /dev/null`

Debugging Containers

Let's use a different Dockerfile.

- `Dockerfile.debug`
- Note the difference! `CMD tail -f /dev/null`

Execute these commands from `/path/to/code/`

- Build and Tag using new Dockerfile

```
docker build -f Dockerfile.debug -t home_example:2.0 .
```

Debugging Containers

Let's use a different Dockerfile.

- `Dockerfile.debug`
- Note the difference! `CMD tail -f /dev/null`

Execute these commands from `/path/to/code/`

- Build and Tag using new Dockerfile

```
docker build -f Dockerfile.debug -t home_example:2.0 .
```

- Run Container as Daemon

```
docker run -d --name create_daemon_ex2 home_example
    :2.0
```

Debugging Containers

Let's use a different Dockerfile.

- `Dockerfile.debug`
- Note the difference! `CMD tail -f /dev/null`

Execute these commands from `/path/to/code/`

- Build and Tag using new Dockerfile

```
docker build -f Dockerfile.debug -t home_example:2.0 .
```

- Run Container as Daemon

```
docker run -d --name create_daemon_ex2 home_example
    :2.0
```

- Explore the new container from within

```
docker exec -it create_daemon_ex2 /bin/sh
```

Today's Examples

You can find these examples at the following link.

`https://github.com/disys-lab/docker_tutorial`

Layering Containers

Execute these commands from `/path/to/code/`

- Build and Tag using new Dockerfile

```
docker build -t home_layered_example:1.0 .
```

- Run Container as Daemon

```
docker run -d --name layered_ex1 home_layered_example
    :1.0
```

Layering Containers

Execute these commands from `/path/to/code/`

- Build and Tag using new Dockerfile

```
docker build -t home_layered_example:1.0 .
```

- Run Container as Daemon

```
docker run -d --name layered_ex1 home_layered_example
    :1.0
```

- Let's try to debug this container
  - Recall the `Dockerfile.debug` file from previous steps

```
docker build -f Dockerfile.debug -t
    home_layered_example:1.0
docker run -d --name layered_ex2 home_layered_example
    :1.0
```

Layering Containers

Execute these commands from `/path/to/code/`

- Build and Tag using new Dockerfile

```
docker build -t home_layered_example:1.0 .
```

- Run Container as Daemon

```
docker run -d --name layered_ex1 home_layered_example:1.0
```

- Let's try to debug this container
  - Recall the `Dockerfile.debug` file from previous steps

```
docker build -f Dockerfile.debug -t home_layered_example:1.0
      .
docker run -d --name layered_ex2 home_layered_example:1.0
```

- Explore the new container from within

```
docker exec -it layered_ex2 /bin/sh
```

Advanced Docker Topics

- File and Data Handling
  - Consume data in files/folders from host.
  - Write or modify files/folders on host.
- Networking with Docker
  - Host Mode
  - Bridge Mode
- Bringing up services
  - Deploy and run many images.
  - Run multiple models at the same time.

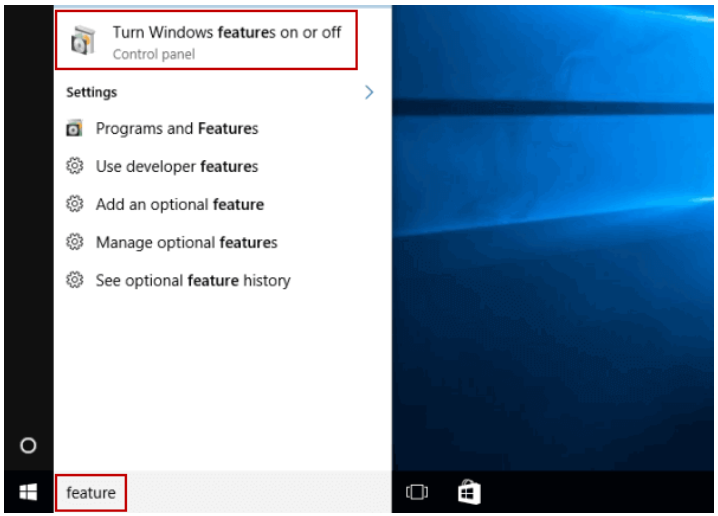
Today's Examples

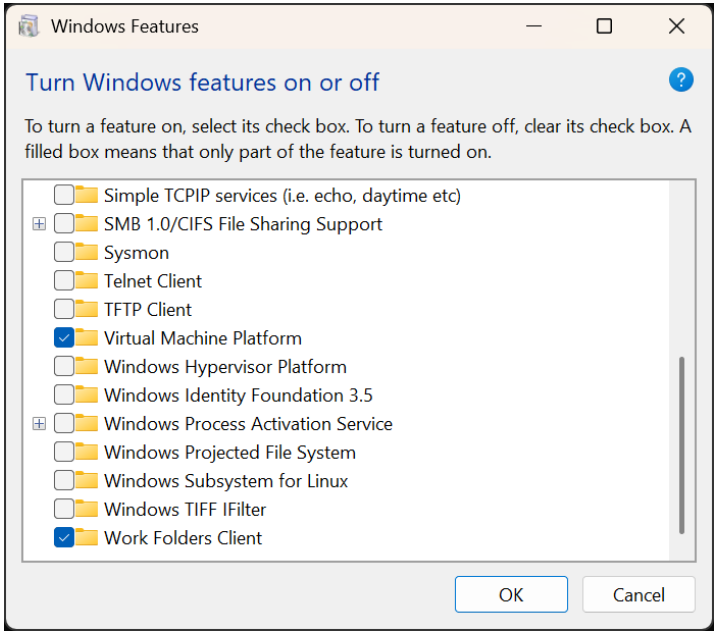
You can find these examples at the following link.

`https://github.com/disys-lab/docker_tutorial`

Windows Installation Instructions

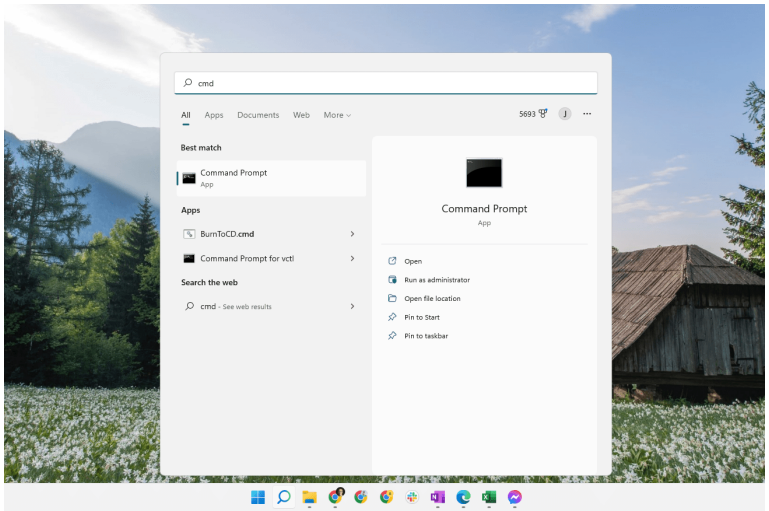
Turning On Windows Features

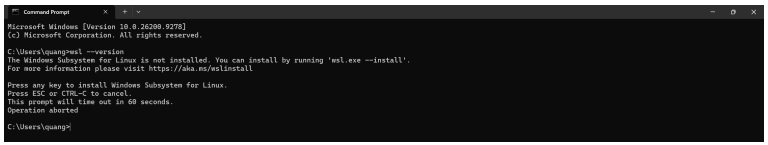




- Search **feature** from the Start menu and open **Turn Windows features on or off**.
- Check **Virtual Machine Platform**, since this is the setting that actually exposes virtualization to WSL2, independent of any BIOS setting.

Checking for WSL from the Command Line





- Search **cmd** and open **Command Prompt**.
- Run `wsl --version`. If WSL is not installed, Windows tells you directly and offers to install it with `wsl --install`.

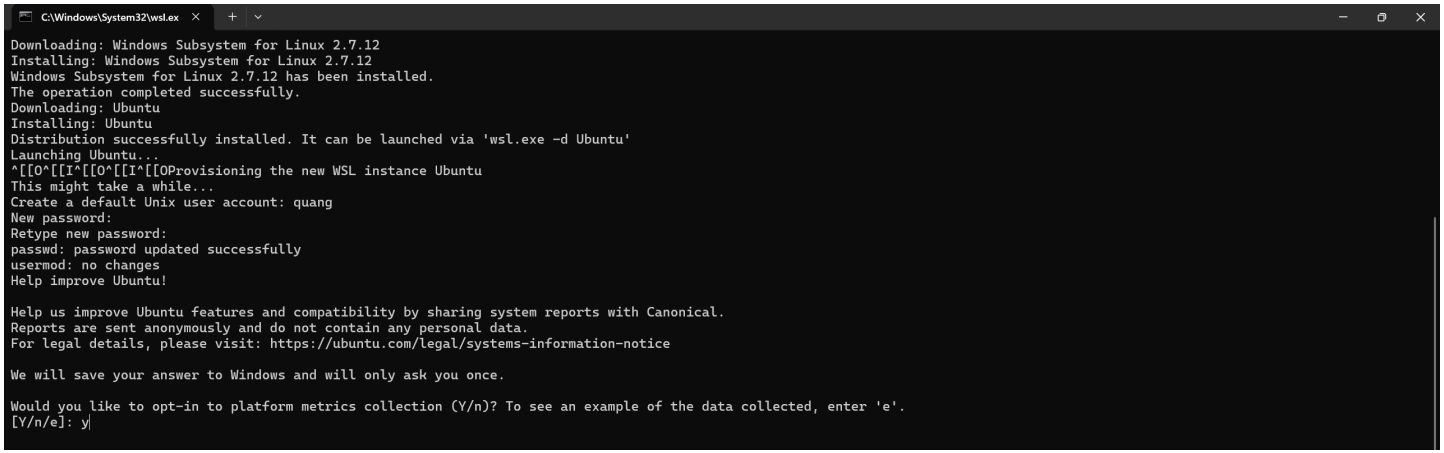
Installing WSL

- Run `wsl --install`.
- Windows downloads and installs the WSL2 kernel update automatically.

Installing Ubuntu

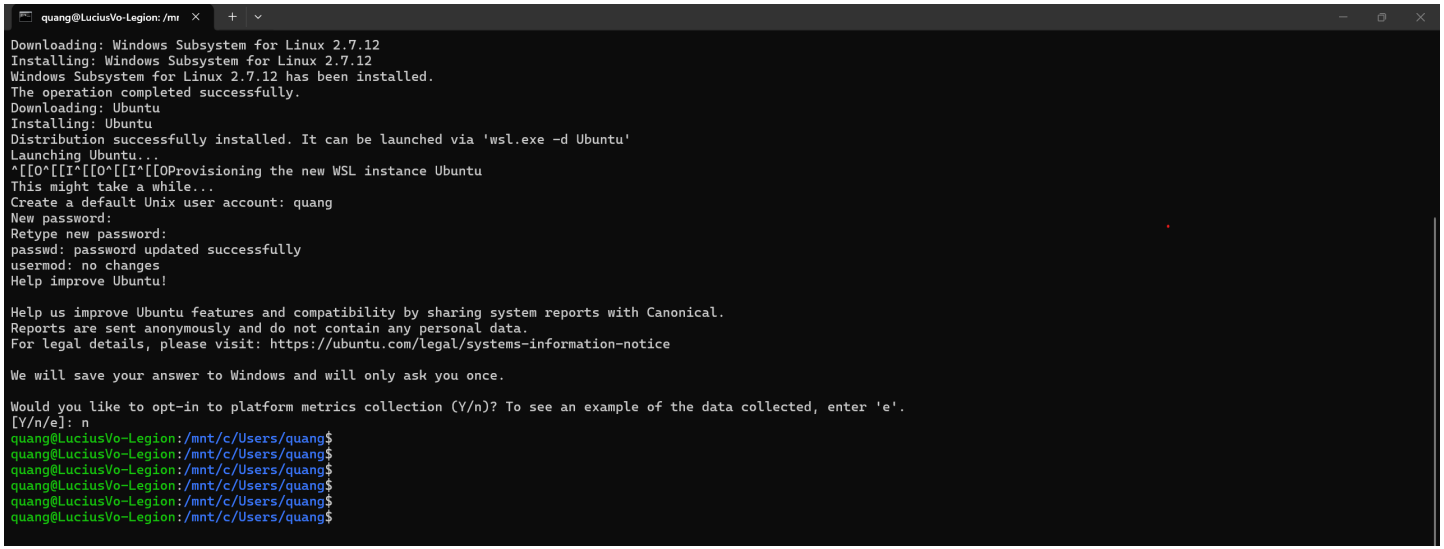
- By default, `wsl --install` also downloads and installs Ubuntu as your Linux distribution.
- Once installed, WSL launches Ubuntu and asks you to create a default Unix user account.

Finishing WSL Setup



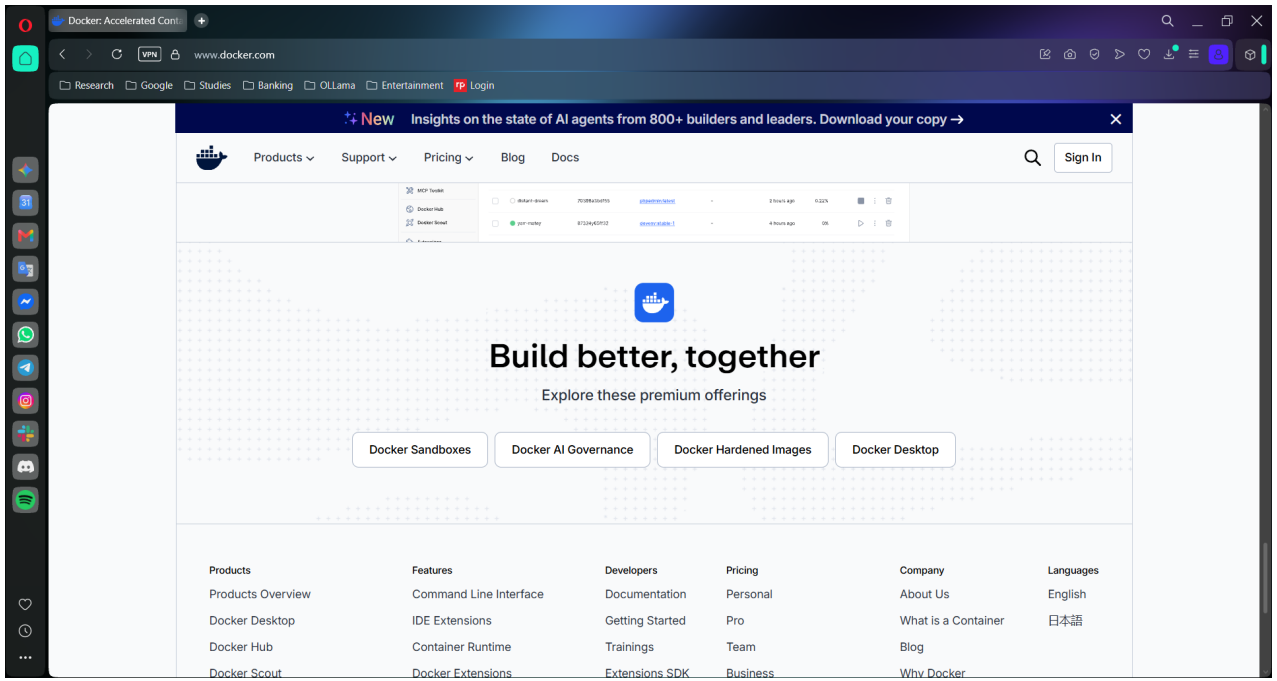
- Set a password for your new Unix user, and answer the Ubuntu metrics-collection prompt however you prefer.

WSL Is Ready



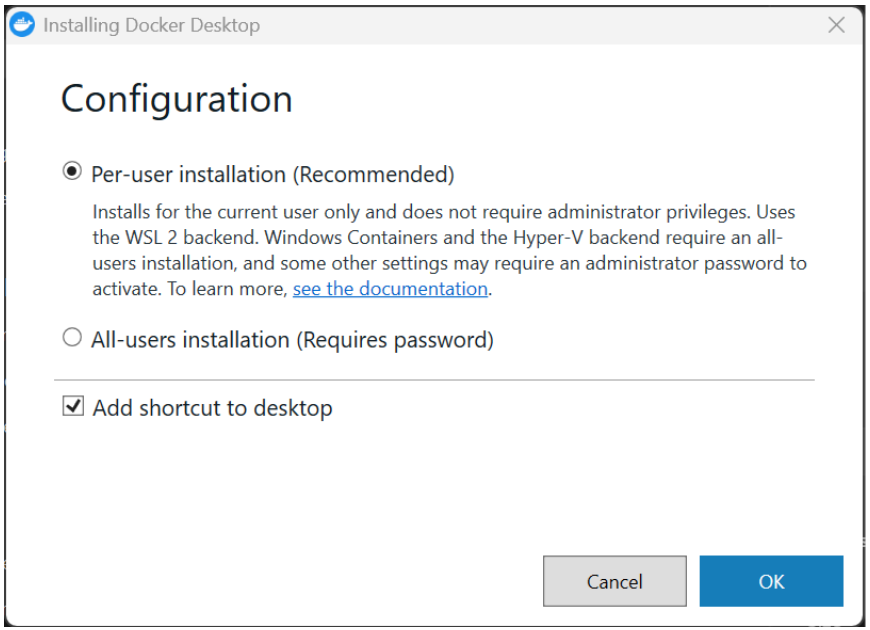
- The prompt changes to your Linux username at a `/mnt/c/...` path, confirming that WSL and Ubuntu are fully set up.
- Docker Desktop can now use WSL2 as its backend.

Downloading Docker Desktop



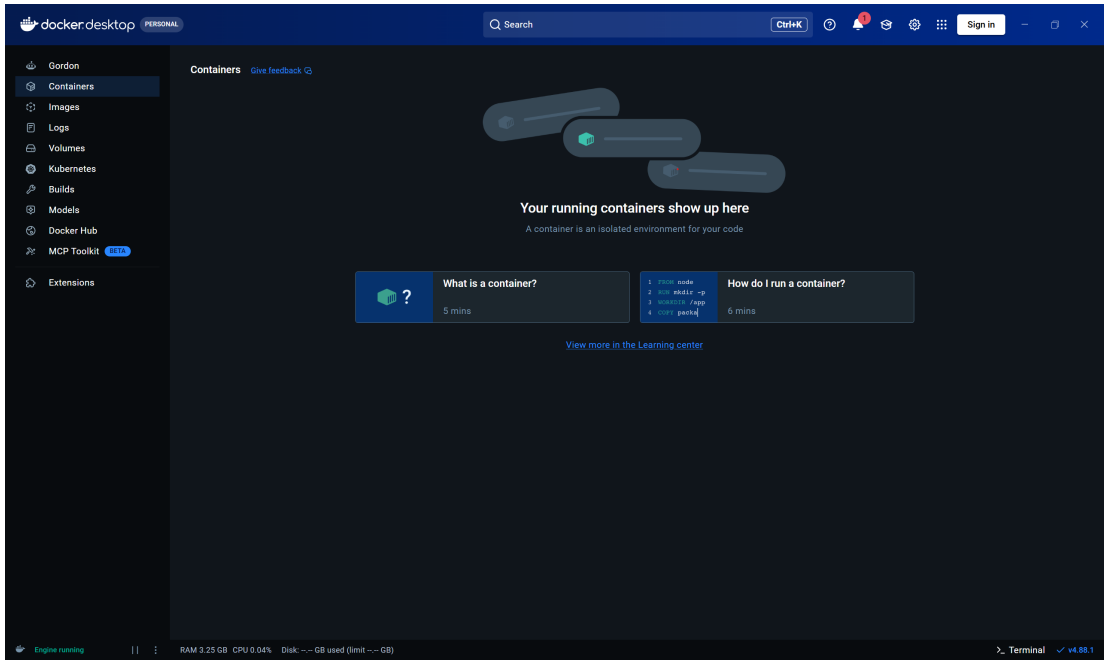
- Go to `https://www.docker.com/products/docker-desktop/` and download the installer for Windows.

Configuring the Installer



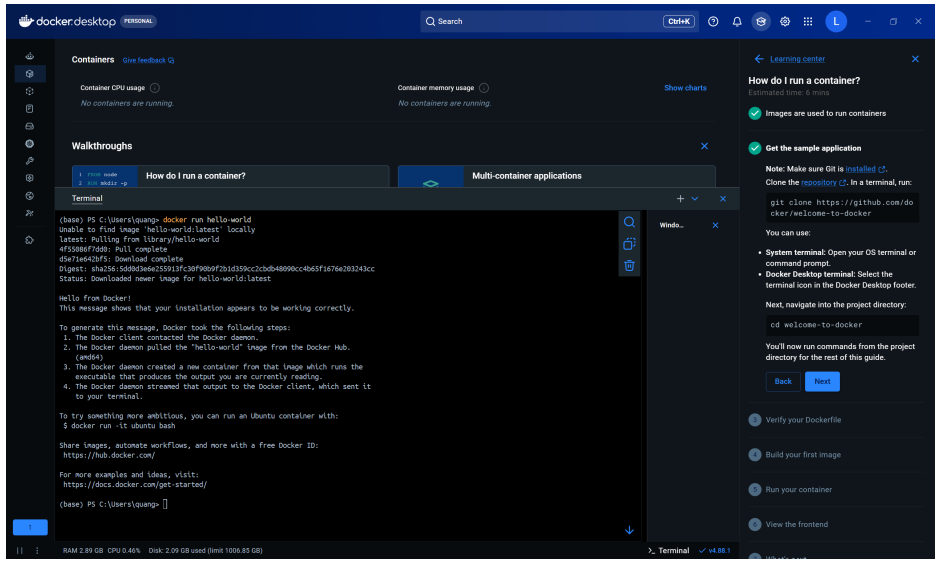
- **Per-user installation** is recommended and does not require administrator privileges.
- It uses the **WSL2 backend**, exactly what we just set up.

Docker Desktop Running



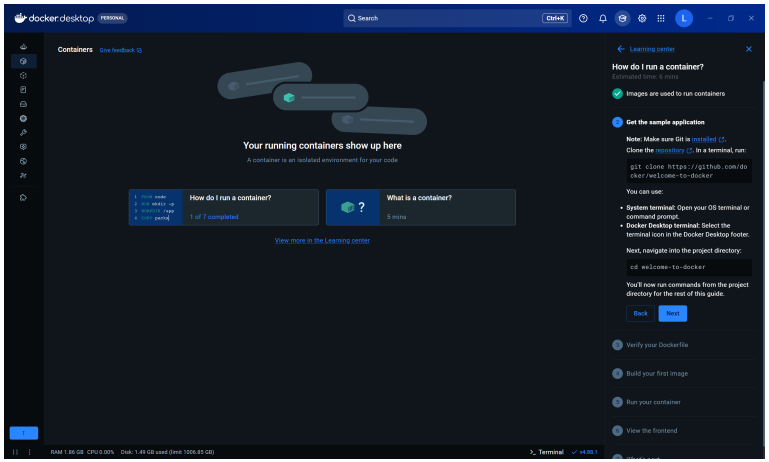
- **"Engine running"** in the bottom-left corner confirms a successful install.
- Compare this to the **"Engine stopped"** problem covered earlier, since enabling Virtual Machine Platform and installing WSL2 is exactly the fix for that.

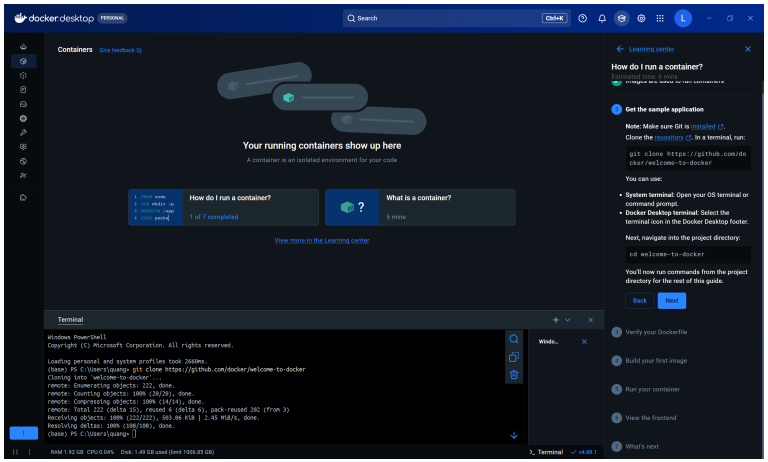
Verifying with `docker run hello-world`



- Running `docker run hello-world` pulls a tiny test image and confirms the whole chain works: client, daemon, and network access to a registry.
- Docker Desktop also has a built-in **Learning Center** panel, open here on the right.

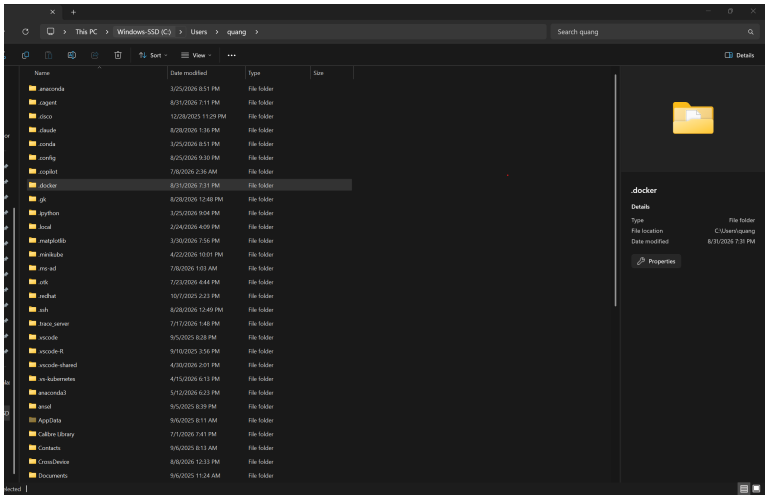
Get the Sample Application

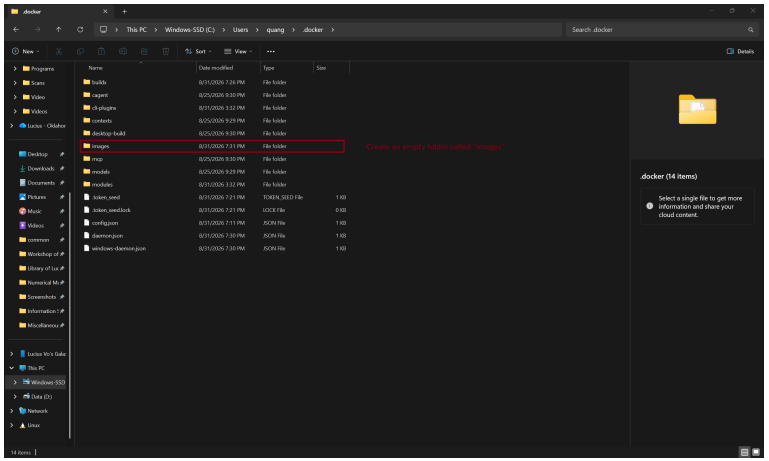




- The Learning Center's **"How do I run a container?"** guide walks through a real sample application.
- Step 2 clones it directly by running `git clone https://github.com/docker/welcome-to-docker`.

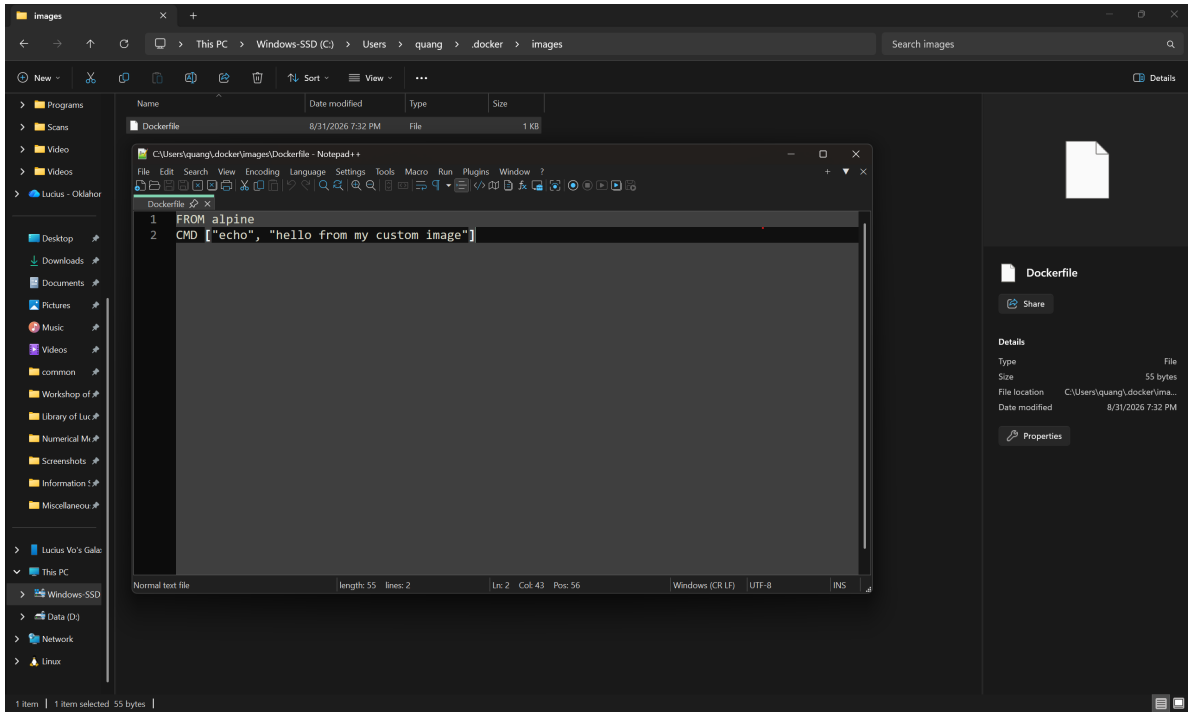
Setting Up a Project Folder





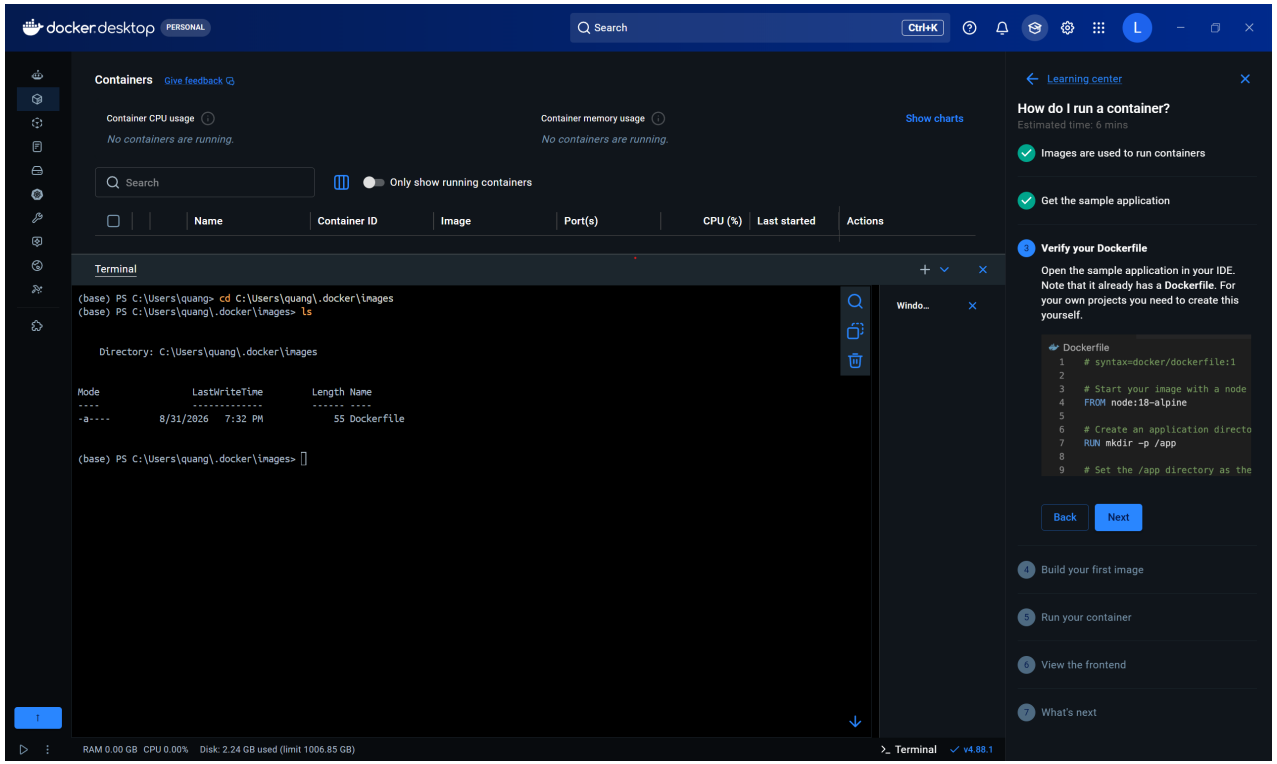
- Unlike the sample application, your own projects need their own folder and `Dockerfile` that you create yourself.
- Here, a new `images` folder is created to hold one.

Writing a Custom Dockerfile



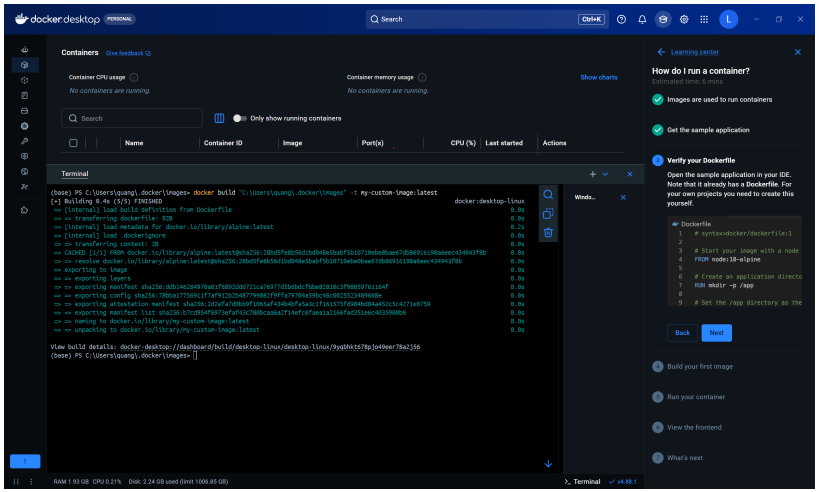
- A minimal `Dockerfile` needs just a base image and a single command.
- `FROM alpine`
  `CMD ["echo", "hello from my custom image"]`

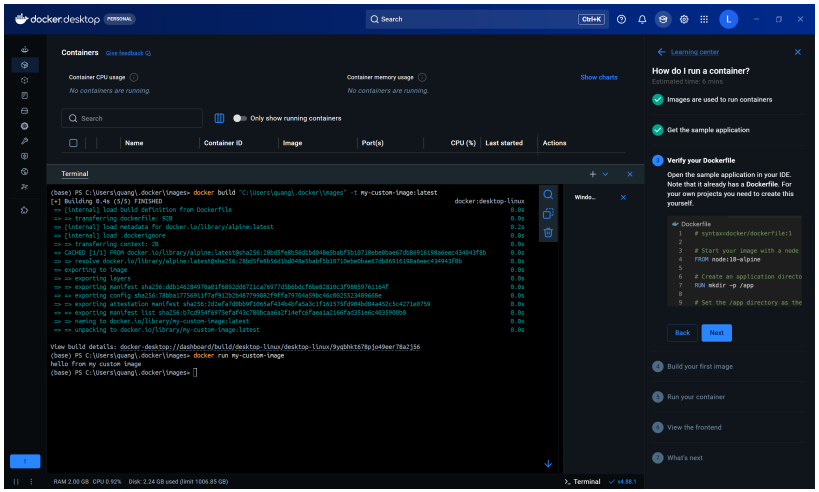
Verifying the Dockerfile



- Back in the terminal, `cd` into the project folder and run `ls` to confirm the `Dockerfile` is there before building.

Building and Running the Custom Image





- `docker build "..." -t my-custom-image:latest` builds the image.
- `docker run my-custom-image` prints "`hello from my custom image`", confirming the custom image works end to end.